

ALUNO: Luis Kamphorst | RA: 240726262

ENTRADA: $(2 + 3) * 4$

```
ENTRANDO em <expr> | lookahead = (  
  ENTRANDO em <term> | lookahead = (  
    ENTRANDO em <factor> | lookahead = (  
      Abrindo parenteses  
      Novo lookahead: 2  
      ENTRANDO em <expr> | lookahead = 2  
        ENTRANDO em <term> | lookahead = 2  
          ENTRANDO em <factor> | lookahead = 2  
            Numero reconhecido: 2  
            Novo lookahead: +  
            SAINDO de <factor> | lookahead = +  
          SAINDO de <term> | lookahead = +  
        Encontrado operador +  
        Novo lookahead: 3  
      ENTRANDO em <term> | lookahead = 3  
        ENTRANDO em <factor> | lookahead = 3  
          Numero reconhecido: 3
```

ALUNO: Luis Kamphorst | RA: 240726262

Programa corrigido!

Resultado: 30

Main.java:8: error: ';' expected

```
    int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e
    virgula
```

^

Main.java:10: error: ')' expected

```
    if (total > 20 { // CORRIGIR 2: feche corretamente o
    parenteses
```

^

Main.java:11: error: unclosed string literal

```
        System.out.println("Programa corrigido!"); // CORRIGIR
    3: feche o texto entre aspas
```

^

3 errors

=== TESTE C2 - identificador personalizado ===

ENTRADA: int nota_lk = 10;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
nota_lk	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===

ENTRADA: int valor# = 1;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

ALUNO: Luis Felipe Kamphorst | RA: 240726262

ENTRADA: total_lk = valor + 62;

LEXEMA	TOKEN

total_lk	IDENTIFICADOR
=	ATRIBUICAO
valor	IDENTIFICADOR
+	SOMA
62	INTEIRO
;	PONTO_E_VIRGULA

ALUNO: Luis Kamphorst | RA: 240726262

ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT
id -> estado 5		
\$ 0 id 5	+id*id\$	REDUCE
F -> id		
\$ 0 F 3	+id*id\$	REDUCE
T -> F		
\$ 0 T 2	+id*id\$	REDUCE
E -> T		
\$ 0 E 1	+id*id\$	SHIFT
+ -> estado 6		
\$ 0 E 1 + 6	id*id\$	SHIFT
id -> estado 5		
\$ 0 E 1 + 6 id 5	*id\$	REDUCE
F -> id		
\$ 0 E 1 + 6 F 3	*id\$	REDUCE
T -> F		
\$ 0 E 1 + 6 T 9	*id\$	SHIFT
* -> estado 7		
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT